

VOLUME 4

Build a Desktop Inside the Browser

*Zustand window state, configuration, focus, drag, maximize, Dock
behavior, and window applications*

A beginner-to-engineer rebuild guide grounded in the real repository

PATTERN > PRACTICE > REPETITION > REFINEMENT > HABIT

Learning Map

Use this volume as a build session, not passive reading. Type the examples, pause at checkpoints, and keep the architecture questions beside your editor.

Chapter	You will learn
1. Model a window	Turn visual behavior into explicit state.
2. Store transitions	Open, focus, minimize, restore, move, and resize.
3. Window wrapper	Connect React lifecycle to GSAP Draggable.
4. Desktop composition	Dock, Home, Navbar, and controls.
5. Add a new app	Build a complete Calculator window using the established pattern.

Chapter 1: Model a Window

Chapter outcome: You will describe a desktop window as data rather than scattered DOM mutations.

Property	Meaning	Why explicit
isOpen	Whether the app participates in the desktop.	Dock and App can make the same decision.
isMinimized	Open but temporarily hidden.	Restore differs from first open.
zIndex	Stacking priority.	Focus becomes deterministic.
position	Current x/y coordinates.	Drag end can persist location.
size	Normal width/height.	Restore can recover user geometry.
sizeMode	normal or full.	Maximize is a state transition, not a CSS guess.
data	Payload passed when opening.	Text/image/code windows can display selected items.

Window configuration is the initial state contract

```
/**
 * PURPOSE:
 * Define desktop shell window defaults.
 * RESPONSIBILITY:
 * Store initial z-index, default sizes, and initial state for each desktop window.
 * USED BY:
 * Desktop shell window store and window wrapper.
 * DEPENDS ON:
 * Desktop window IDs.
 * SHOULD NOT HANDLE:
 * Rendering windows, opening windows, portfolio content, or mobile shell behavior.
 * SCALING NOTES:
 * Add new desktop windows here only when they participate in desktop shell window management.
 */

export const INITIAL_Z_INDEX = 1000;

export const DEFAULT_WINDOW_SIZES = {
  finder: { width: 900, height: 650 },
  contact: { width: 600, height: 500 },
  resume: { width: 800, height: 900 },
  safari: { width: 1200, height: 700 },
  photos: { width: 1000, height: 650 },
  terminal: { width: 700, height: 500 },
  vscode: { width: 1100, height: 700 },
  txtfile: { width: 500, height: 600 },
  imgfile: { width: 600, height: 700 },
};
```

```

const createWindowConfig = (size) => ({
  isOpen: false,
  isMinimized: false,
  sizeMode: "normal",
  zIndex: INITIAL_Z_INDEX,
  data: null,
  size,
  position: null,
  lastNormalSize: size,
  lastNormalPosition: null,
});

export const WINDOW_CONFIG = {
  finder: createWindowConfig(DEFAULT_WINDOW_SIZES.finder),
  contact: createWindowConfig(DEFAULT_WINDOW_SIZES.contact),
  resume: createWindowConfig(DEFAULT_WINDOW_SIZES.resume),
  safari: createWindowConfig(DEFAULT_WINDOW_SIZES.safari),
  photos: createWindowConfig(DEFAULT_WINDOW_SIZES.photos),
  gallery: createWindowConfig(DEFAULT_WINDOW_SIZES.gallery),
  terminal: createWindowConfig(DEFAULT_WINDOW_SIZES.terminal),
  vscode: createWindowConfig(DEFAULT_WINDOW_SIZES.vsCode),
  txtfile: createWindowConfig(DEFAULT_WINDOW_SIZES.txtfile),
  imgfile: createWindowConfig(DEFAULT_WINDOW_SIZES.imgfile),
};

```

Chapter 2: Store Transitions

Chapter outcome: You will read Zustand updates and understand immutable nested state.

Open, close, and focus transitions

```

    Math.round(navHeight + (availableHeight - windowSize.height) / 2)
  ),
};
};

const useWindowStore = create((set) => ({
  windows: WINDOW_CONFIG,

  openWindow: (name, data = null) => {
    currentZIndex += 1;

    set((state) => {
      const current = state.windows[name];
      if (!current) return state;

      return {
        windows: {
          ...state.windows,
          [name]: {
            ...current,
            isOpen: true,
            isMinimized: false,
            data,
            zIndex: currentZIndex,
            position: getCenteredPosition(name, current.size),
          },
        },
      },
    });
  },
});

```

```

    });
  },

  closeWindow: (name) => {
    set((state) => ({
      windows: {
        ...state.windows,
        [name]: {
          ...state.windows[name],
          isOpen: false,
          isMinimized: false,
        },
      },
    }));
  },

  focusWindow: (name) => {
    currentZIndex += 1;

    set((state) => ({
      windows: {
        ...state.windows,
        [name]: {

```

Zustand's set function receives the previous state and returns the next state. The spread operators create a new windows object and a new record for one window while preserving every unrelated window. React subscribers can detect the changed references and re-render only where needed.

Hidden global state: currentZIndex lives at module scope, outside the store object. It works for one browser session, but it is less visible to tests, reset tools, and devtools. A future refinement could store nextZIndex inside Zustand if those needs become important.

Opening a desktop app

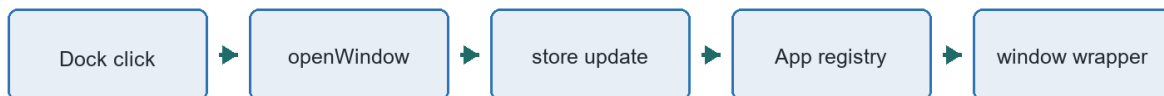


Figure 1. State drives rendering; the Dock does not create DOM windows directly.

Chapter 3: The Window Wrapper

Chapter outcome: You will understand higher-order components, refs, layout effects, and drag cleanup.

A higher-order component returns a managed component

```

import { DEFAULT_WINDOW_SIZES } from "../config/windowConfig";
import useWindowStore from "../store/windowStore";

```

```

gsap.registerPlugin(Draggable);

const windowWrapper = (Component, windowKey) => {
  const Wrapped = (props) => {
    const store = useWindowStore();

    const {
      focusWindow,
      windows,
      setWindowPosition,
      setWindowSize,
      setSizeMode,
    } = store;

    const {
      isOpen,
      sizeMode = "normal",
      zIndex = 1000,
      position,
      size,
    } = windows?.[windowKey] || {};

    const ref = useRef(null);
    const originalSizeRef = useRef(null);
    const originalPositionRef = useRef(null);
    const prevSizeModeRef = useRef(sizeMode);

    useEffect(() => {
      if (isOpen && sizeMode === "normal") {
        originalSizeRef.current =
          size || DEFAULT_WINDOW_SIZES[windowKey] || { width: 800, height: 600 };

```

WHAT A higher-order component is a function that receives a component and returns another component.

WHY All desktop apps need the same frame mechanics, so the behavior is centralized once.

WHEN Use it when several components require the same wrapper contract and hooks cannot alone provide the surrounding DOM.

WHERE The generic mechanics live in desktop-shell/hoc; app content stays in its own window file.

HOW The wrapper reads store state, holds a DOM ref, applies GSAP geometry, creates a Draggable instance, and kills it during cleanup.

Maximize and restore synchronization

```

    originalPositionRef.current = position || { x: 0, y: 0 };
  }
}, [isOpen, size, position, sizeMode]));

useLayoutEffect(() => {
  if (window.innerWidth <= 768) return;

  const el = ref.current;
  if (!el || !isOpen) return;

  const prevMode = prevSizeModeRef.current;

  if (prevMode !== sizeMode) {
    if (sizeMode === "full") {

```

```

    gsap.to(el, {
      x: 0,
      y: 0,
      width: window.innerWidth,
      height: window.innerHeight,
      borderRadius: 0,
      duration: 0.34,
      ease: "power3.inOut",
    });
  }

  if (sizeMode === "normal") {
    const restoreSize =
      originalSizeRef.current ||
      DEFAULT_WINDOW_SIZES[windowKey] || { width: 800, height: 600 };

    const restorePos = originalPositionRef.current || { x: 0, y: 0 };

    gsap.to(el, {
      x: restorePos.x,
      y: restorePos.y,
      width: restoreSize.width,
      height: restoreSize.height,
      borderRadius: 10,
      duration: 0.34,
      ease: "power3.inOut",
    });
  }
} else {
  if (sizeMode === "normal") {
    gsap.set(el, {
      x: position?.x || 0,
      y: position?.y || 0,
      width: size?.width || DEFAULT_WINDOW_SIZES[windowKey]?.width || 800,
      height:
        size?.height || DEFAULT_WINDOW_SIZES[windowKey]?.height || 600,
    });
  }
}

if (sizeMode === "full") {
  gsap.set(el, {
    x: 0,
    y: 0,
    width: window.innerWidth,
    height: window.innerHeight,
    borderRadius: 0,
  });
}
}
}

```

useLayoutEffect runs after DOM changes but before the browser paints. That timing prevents a visible flash at the old geometry. useEffect is sufficient for remembering values; useGSAP owns the imperative drag instance and cleanup.

Browser implication: Transforms such as x and y usually avoid document reflow during drag and are efficient for animation. Width and height changes can trigger layout, so they are reserved for mode transitions and resizing rather than every pointer movement.

Chapter 4: Desktop Composition

Chapter outcome: You will connect Dock, Home, Navbar, controls, and app windows without mixing responsibilities.

Part	Primary job	Dependency
Dock	Launch, restore, and focus apps.	Window store and desktopApps config.
Home	Desktop icons and wallpaper surface.	Locations and openWindow.
Navbar	Top desktop chrome and clock/menu state.	Window store and React time state.
WindowControls	Close, minimize, maximize commands.	Window store.
App.jsx registry	Render open non-minimized app components.	Window records and lazy imports.
windowWrapper	Generic movement and geometry.	Store, config, GSAP.

Notice the rhythm: configuration describes, stores transition, components request, App composes, and the wrapper applies generic mechanics. Repeating that pattern makes a new desktop app predictable.

Chapter 5: Add a New Calculator App

Chapter outcome: You will be able to extend the desktop system without inventing a second architecture.

1. Add calculator geometry to windowConfig.js.
2. Add its icon and id to desktopApps.js.
3. Create windows/Calculator.jsx with local calculator state.
4. Wrap it with windowWrapper(Calculator, 'calculator').
5. Lazy-import it in app/App.jsx.
6. Register desktopRenderWindow(Calculator, 'calculator').
7. Add focused CSS using a calculator prefix.
8. Verify open, focus, minimize, restore, drag, and maximize.

Ownership: The calculator's expression and result are app-specific local state. Window position and focus remain desktop-shell state. Do not put calculator arithmetic inside the global window store.

Checkpoint

- Beginner: Explain why isOpen and isMinimized are separate.

- Intermediate: Trace a drag from pointer press to persisted position.
- Advanced: Design keyboard focus and accessibility rules for ten simultaneous windows.